

Evaluating your Agent

Is your Agent doing it's job?

Abhijith Neerkaje & Ajay Shenoy

Why Agents Make Mistakes?

Gulf of Comprehension

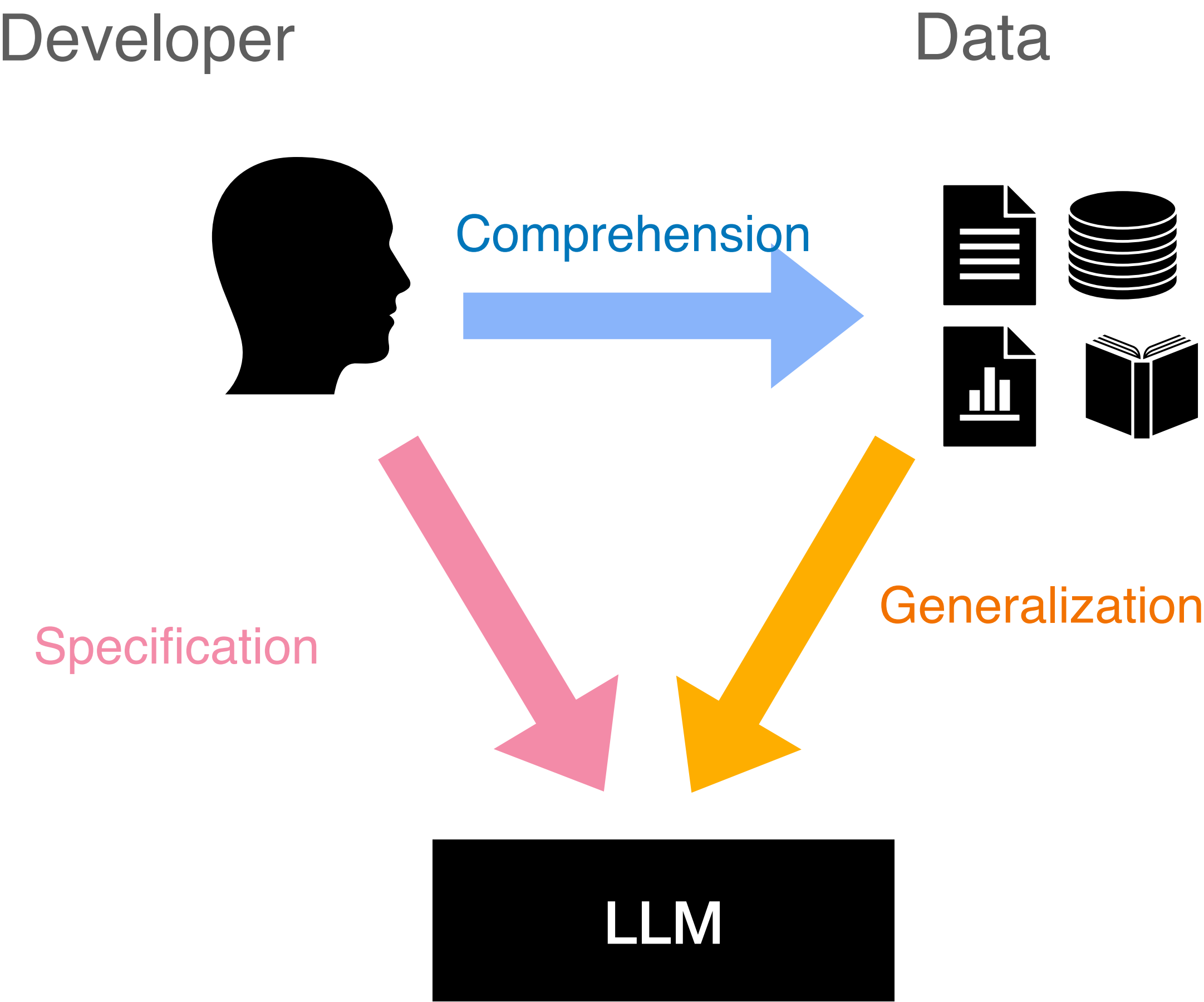
Developers struggle to fully understand their input data and how the pipeline behaves across different scenarios

Gulf of Specification

User intent is often loosely captured in prompts, leading to ambiguous task definitions and inconsistent results

Gulf of Generalization

Models behave unpredictably across different inputs, with biases and errors emerging in production



Why Traditional Testing Falls Short in Evaluating Agents

Non-Deterministic

Unlike traditional software, LLMs produce different outputs for identical inputs

Subjective Quality

Responses vary in quality based on context, user intent, and domain expertise

Multifaceted Assessment

Success requires evaluating accuracy, relevance, safety, and user satisfaction simultaneously

These characteristics make Agentic AI evaluation fundamentally different from classic software testing and ML testing approaches.

What Agentic Evaluation Needs

- **No Universal Solutions:** Each pipeline requires bridging the Three Gulfs anew
- **Context-Specific Metrics:** Success depends on your specific goals, not generic benchmarks
- **Evolving Requirements:** Evaluation needs change as systems and users mature
- **Resource Intensive:** Meaningful evaluation requires significant time and domain expertise
- **Multiple components:** Each component of agent requires separate evaluation



Generic benchmarks often fail to capture real-world performance issues that matter most to your users.

Evaluation Metrics Overview

Core Purpose

Provide systematic measurements of LLM pipeline quality and reliability across different use cases

Two Main Categories

Reference-based metrics: Compare LLM output against known, ground-truth answers. The known answer is called a "reference" or "golden" output.

Reference-free metrics: Evaluate output based on inherent properties or rule adherence without specific "golden" answers.

Your choice depends on what you're evaluating and available resources. Both approaches have distinct use cases and trade-offs that impact implementation strategy.

Evaluation by components of Agents

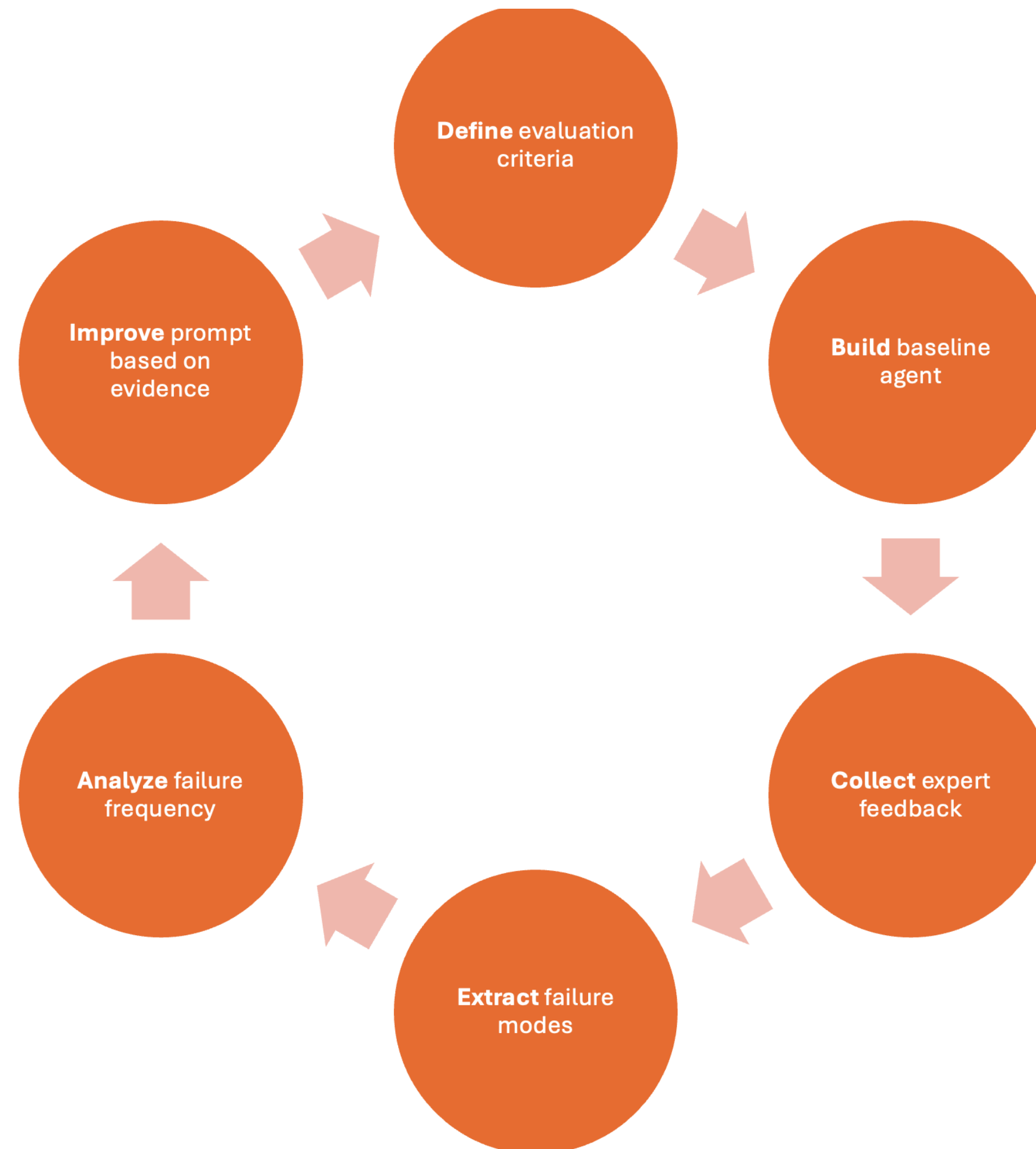
Component	Metric Category	Example Metrics	Example
System	Operational Metrics	- Latency, Cost	Avg response time = 900ms Cost per 1000 queries = \$1.20
Prompting	Performance Metrics	- Instruction Adherence Score/ Accuracy - Clarity - Bias Score	Adherence Score = 92/100 Ambiguity detected in 5% of prompts
RAG	Retrieval & Generation	- Precision@k, Recall@k - Hallucination Rate/Groundedness - Retrieval Time - Cache Hit Rate	Precision@3 = 78% Hallucination Rate = 4% Cache hit = 65%
Planning System	Task Execution	- Task Completion Rate - Steps to Completion - Sub-goal Success Rate	Task Completion = 85% Average steps = 4.2 Sub-goal success = 70%
Tool Use	Tool Invocation Metrics	- Tool Match Accuracy - Tool Latency - Tool Error Rate	Match Accuracy = 88% Error Rate = 2.5% Tool Latency = 1.3s

Tips for generating synthetic queries

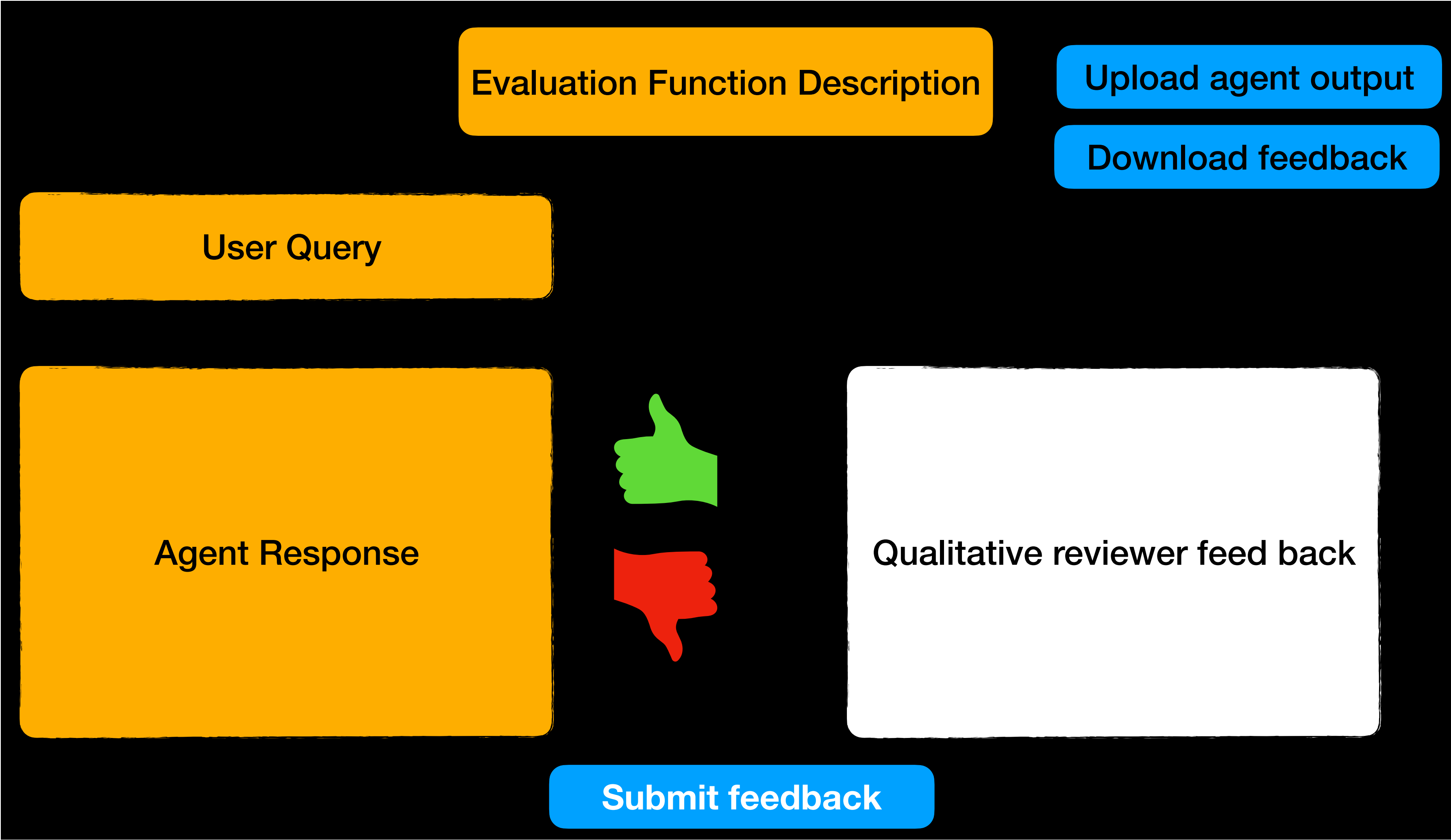
- **Common Pitfall:** Asking LLMs for queries without structure → repetitive, generic outputs.
- **Step 1: Define Dimensions** – Capture user behavior variations, e.g.:
 - Product Search: Category, Price Range, Brand
 - Customer Support: Issue Type, Mood, Order Context
- **Step 2: Identify Failures** – Target likely failure modes via testing or observation.
- **Step 3: Manual Tuples** – Create ~20 tuples combining one value per dimension, e.g., (Electronics, Premium, Specific Brand).
- **Step 4: Scale with LLM**
 - Generate more structured tuples.
 - Convert tuples to natural language queries, e.g., "Show me the latest high-end Bose headphones under ₹30,000."
- **Tip:** Separate tuple generation and query phrasing to avoid repetition

Error Analysis

Error analysis is the systematic process of examining model failures to identify recurring patterns and use them to guide targeted improvements

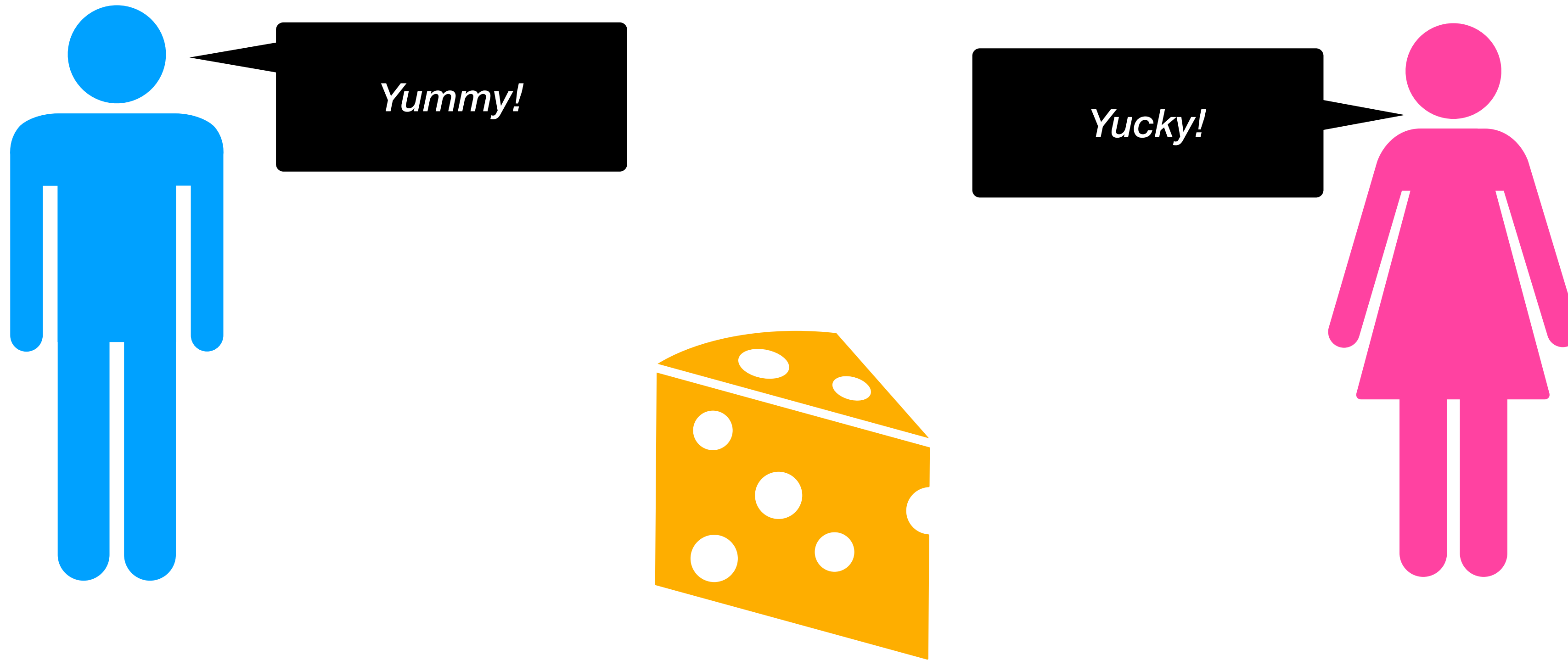


Let us design a custom data annotator to speed up error analysis



Task: Open Qwen and write a prompt to design this front end. Upload the csv and annotate data.

How to deal with subjectivity of reviewers?



Solution : Let us ask the neighbors! Use collaborative evaluation or ask a benevolent dictator

Collaborative Evaluation Process Flow

A systematic approach to building consensus and ensuring high-quality labeled datasets through collaborative refinement and iterative improvement.



Assemble Team

Gather some domain experts and key stakeholders to ensure diverse perspectives and comprehensive coverage of annotation requirements.



Draft Initial Rubric

Define clear annotation criteria with specific pass/fail examples that illustrate edge cases and decision boundaries for consistent labeling.



Select Shared Dataset

Choose 20-50 diverse, challenging examples that represent the full spectrum of cases annotators will encounter in production.



Independent Annotation

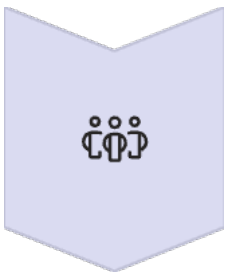
Each team member labels the dataset separately without discussion to capture initial interpretations and identify potential ambiguities.

Collaborative Evaluation Process Flow - Contd



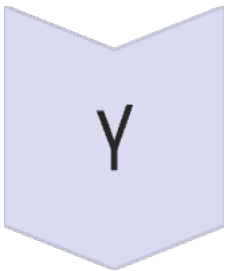
Measure Agreement

Compute Inter-Annotator Agreement (IAA) metrics to quantify consensus levels and pinpoint areas requiring clarification or refinement.



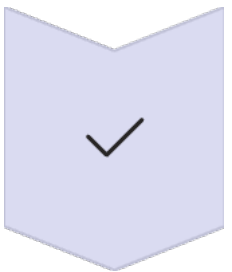
Alignment Session

Facilitate structured discussions around disagreements, explore reasoning behind different interpretations, and collaboratively refine the rubric.



Revise & Iterate

Update the rubric based on insights, re-label the dataset with improved guidelines, and re-measure IAA to track progress toward consensus.



Finalize Rubric & Gold Set

Deliver a comprehensive, documented rubric alongside a consensus-labeled golden dataset ready for training and quality assurance.

This iterative workflow ensures annotation consistency by building consensus through structured collaboration, ultimately producing high-quality labeled data that serves as the foundation for reliable machine learning models.

Why Measure Expected Agreement by Chance?

Understanding the reliability of evaluation metrics requires looking beyond surface-level agreement rates. When two evaluators align on their assessments, it's crucial to determine whether that agreement stems from genuine consensus or merely statistical coincidence.

The Problem with Raw Agreement

Why Raw Numbers Mislead

Two evaluators frequently agree even without true alignment. Shared biases—like the tendency to mark most responses as acceptable—artificially inflate agreement scores. Raw agreement percentages don't provide a reliable evaluation signal for measuring annotation quality.

Expected Agreement by Chance

How often would evaluators agree if they labeled independently, based only on their individual label tendencies?

This metric reveals the baseline agreement rate that occurs purely through statistical probability, not genuine evaluator consensus.

A Real-World Example

- 1 — Annotator Tendencies
Annotator 1: 60% Acceptable | 40% Unacceptable
Annotator 2: 65% Acceptable | 35% Unacceptable

- 2 — Calculate Chance
Acceptable: $0.60 \times 0.65 = 0.39$
Unacceptable: $0.40 \times 0.35 = 0.14$
Total chance agreement = 53%

- 3 — True Alignment Revealed
If raw agreement is 80%, only 27% reflects genuine consensus after removing the 53% chance agreement baseline.

Key Takeaway: Cohen's Kappa removes chance agreement to reveal how reliable your human or LLM-as-judge evaluations really are. It's the difference between apparent agreement and meaningful evaluator alignment.

Key Take Aways

- 3 Gulfs [Comprehension, Specification and Generalization]
- Challenges of evaluating agent responses
- Reference free and reference based metrics
- Component wise evaluation of agents
- How generate synthetic data
- Error analysis
- Custom annotators to help with error analysis
- How to deal with subjectivity

Appendix

Reference-Based Metrics

Definition & Approach

Compare LLM output against known, ground-truth answers. The known answer is called a "reference" or "golden" output.

Requires preparing correct reference answers in advance for test data - similar to having an official answer key for grading tests.



Valuable during development cycle as unit tests and typically used in offline testing stages

Reference-Based Metrics

Simple Examples

Exact String Matching

Check if LLM output exactly matches reference string.
Useful for short-answer extraction tasks where precision is critical.

Keyword Verification

Verify output contains specific keywords expected from reference. More flexible than exact matching, allowing for variations.

Both approaches work well for straightforward, objective comparisons where answers have limited acceptable variations.

Complex Examples

Execution-Based Evaluation

Execute generated SQL and compare output result/table to known correct result.
Run LLM-generated code against pre-defined unit tests.

Hybrid Approaches

Execute generated code first, then use oracle (LLM judge or human) to evaluate runtime behavior and alignment with intended reference behavior.

Reference-Free Metrics

Evaluate output based on inherent properties or rule adherence without specific "golden" answers.

Crucial for subjective, creative, or open-ended outputs where multiple valid responses exist.

Examples:

Text Summarization



Check if summary introduces speculative claims not in original source. Verify no opinions added beyond source material.

Customer Service Chatbot



Assess whether bot refrains from definitive medical/financial advice. Ensure guidance toward expert consultation.

Code Generation



Verify code includes explanatory comments for non-trivial logic. Check avoidance of deprecated functions.

Marketing Copy



Ensure clear call-to-action present. Exclude overly aggressive sales phrases inconsistent with brand guidelines.



Goal: Check desired properties of output directly

Reference-Free Metrics- Key Insights & Strategic Considerations

Application-Specific Quality



Reference-free qualities are highly context-dependent and require careful definition based on use case requirements.

Production Reality



Generating real-time ground truth often impractical in production, making self-contained checks vital for system reliability.

Strategic Selection

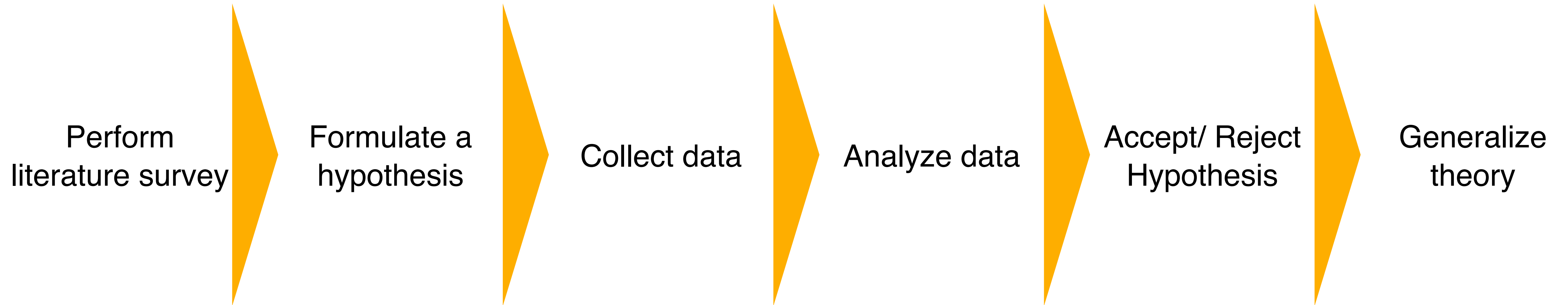


Choose evaluation strategy based on available resources, specific quality requirements, and production constraints.

No one-size-fits-all approach - evaluation method depends heavily on what you're evaluating

How we usually reason about quantitative data?

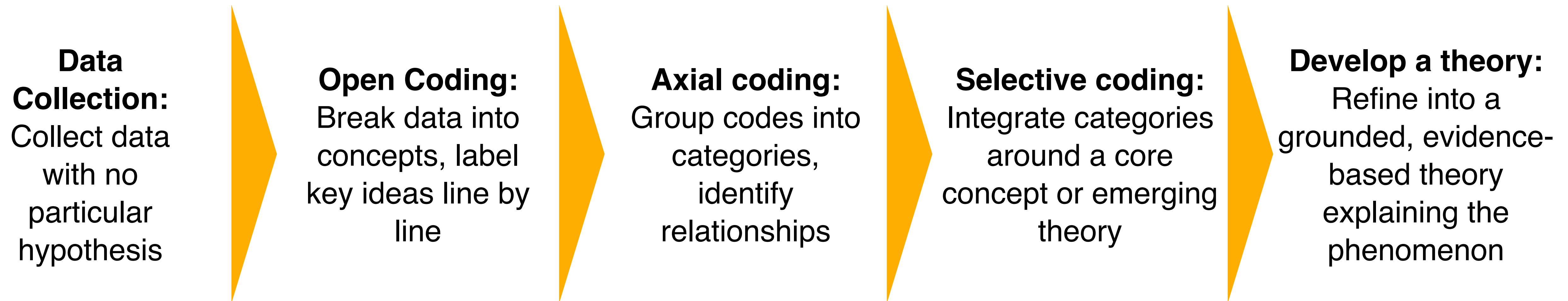
Traditional Approach to build a theory about a system or a process



Example: Let us test this hypothesis “Faster delivery increases customer trust scores by 20%.”

Not Suitable for qualitative data !

Reasoning about qualitative data : Grounded theory approach



Error Analysis is based on grounded theory approach.

Error analysis is the structured investigation of an agent's mistakes to uncover the root causes across its reasoning and actions, so we can improve the agent rather than just score it.

Class activity: Personal Portfolio Manager Agent

You are testing an AI agent that helps users manage their investments.

Users often ask vague or repetitive questions, so you need to create **structured test queries** to cover different user behaviors.

Step 1: Dimensions

- **Asset Type:** Stocks, Mutual Funds, Crypto, Bonds
- **Goal:** Growth, Income, Risk Management
- **Action:** Buy, Sell, Hold, Rebalance, Compare

Step 2: Tuples (examples)

- (Stocks, Growth, Buy)
- (Crypto, Risk Management, Sell)
- (Mutual Funds, Income, Compare)

Step 3: Convert Tuples into Queries

- “Which tech stocks should I buy for long-term growth?”
- “Should I sell my Bitcoin to reduce portfolio risk?”
- “Compare dividend-focused mutual funds that provide steady income.”

Exercise Prompt for Students:

Write code to use the dimensions → tuples → queries method to generate 50 test queries for a portfolio manager agent. Make sure each query sounds like something a real investor would ask.